

Securitatea Sistemelor Informatice



- Curs 10.2 - Schimb de chei autentificat. TLS

Adela Georgescu

Facultatea de Matematică și Informatică
Universitatea din București
Anul universitar 2025-2026, semestrul II

Schimb de chei autentificat - AKE (Authenticated Key Exchange)

- ▶ Schimbul de chei Diffie-Hellman permite stabilirea unei chei de sesiune dar nu este autentificat - susceptibil la un atac de tip man-in-the-middle
- ▶ Un protocol autentificat permite stabilirea unui *chei de sesiune* comună între P și Q, la finalul căruia fiecare dintre P și Q știe foarte clar cu cine a stabilit cheia de sesiune.
- ▶ Este necesară prezența unei terțe părți de încredere - TTP (trusted third party) - cu care utilizatorul efectuează un *protocol de înregistrare* și stabilește propria cheie secretă *long term secret key*

Schimb de chei autentificat - AKE (Authenticated Key Exchange)

- ▶ **TTP** - rol de autoritate de certificare (CA) - emite certificate ce leaga o *identitate* (entitate) din viata reala de o *cheie publică*.
- ▶ **Instanță** (a utilizatorului) - o executie a protocolului de catre utilizator.
- ▶ **Adversarul** - are control complet asupra rețelei

Schimb de chei autentificat - AKE (Authenticated Key Exchange)

- ▶ Cea mai simplă construcție de AKE pare a fi schema de identificare + schimb de chei
- ▶ Insa nici una din cele doua variante de mai jos nu este sigură

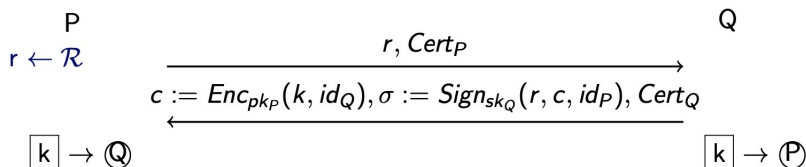
Varianta 1

- ▶ P se identifică în fața lui Q
- ▶ Q se identifică în fața lui P
- ▶ P și Q generează o cheie comună

Varianta 2

- ▶ P și Q generează o cheie secretă și implementează un canal de comunicare securizat
- ▶ P se identifică în fața lui Q folosind acest canal
- ▶ Q se identifică în fața lui P folosind acest canal

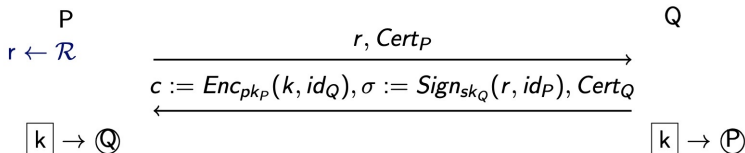
AKE₁ - prima varianta sigură



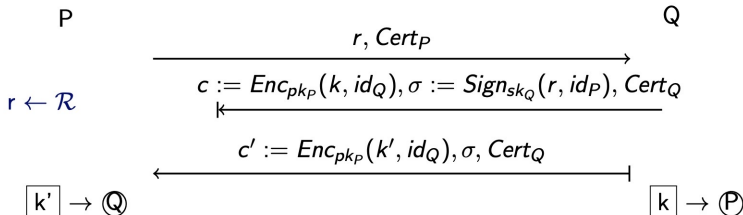
- ▶ $Cert_P$ reprezinta certificatul lui P care leaga identitatea id_P de cheile publice pentru criptare și semnatura
- ▶ $\mathcal{R}$ este o multime din care se genereaza numere aleatoare nonce
- ▶ Notatia $\boxed{k} \rightarrow \mathbb{Q}$ indica faptul ca la finalizarea protocolului, P detine cheia de sesiune k si considera ca a vorbit cu Q.

AKE₁ - variatii nesigure

Varianta 1 : c nu e semnat



Atac

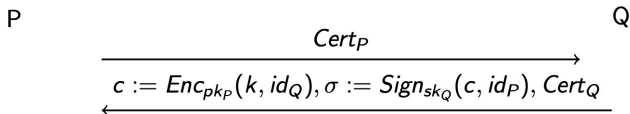


AKE₁ atac varianta 1

- ▶ In schema, notatia $| \leftarrow$ indica un mesaj blocat de adversar iar $\leftarrow |$ indica un mesaj generat de adversar
- ▶ k' este o cheie de sesiune aleasa de adversar

AKE₁ - variatii nesigure

Varianta 2 : r nu e semnat - replay attack

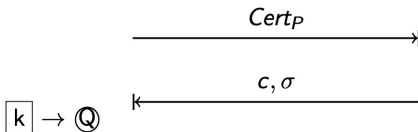


$\boxed{k} \rightarrow \textcircled{Q}$

$\boxed{k} \rightarrow \textcircled{P}$

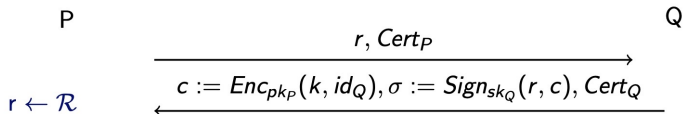
...

P



AKE_1 - variatii nesigure

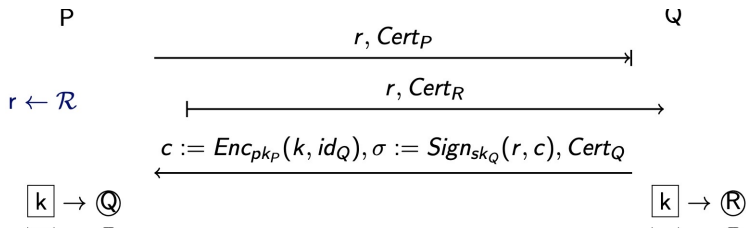
Varianta 3 : id_P nu e semnat



$\boxed{k} \rightarrow \textcircled{Q}$

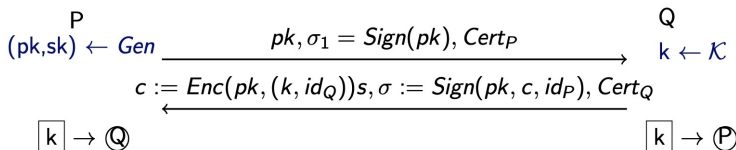
$\boxed{k} \rightarrow \textcircled{P}$

Atac



AKE imbutatit

Perfect forward secrecy - daca o cheie long term este compromisa, atunci cheile de sesiune generate anterior raman secrete.



Protocolul este PFS sigur pentru ca se folosesc chei diferite pentru criptare si semnare

One-sided AKE - Q (server-ul) are un certificat iar P (clientul) nu are

Studiu de caz: TLS - Transport Layer Security

- ▶ Este un protocol folosit de browser-ul web de fiecare data cand ne conectam la un browser folosind `https`

Studiu de caz: TLS - Transport Layer Security

- ▶ Este un protocol folosit de browser-ul web de fiecare data cand ne conectam la un browser folosind `https`
- ▶ primele versiuni se numeau SSL - Secure Sockets Layer - dezvoltat de Netscape (1995) - SSL 3.0, cea mai cunoscută versiune
- ▶ TLS 1.0 apare în 1999, TLS 1.1 în 2006, TLS 1.2 în 2008 și versiunea actuală, sigura și eficientă TLS 1.3 în 2018

Studiu de caz: TLS - Transport Layer Security

- ▶ Este un protocol folosit de browser-ul web de fiecare data cand ne conectam la un browser folosind https
- ▶ primele versiuni se numeau SSL - Secure Sockets Layer - dezvoltat de Netscape (1995) - SSL 3.0, cea mai cunoscută versiune
- ▶ TLS 1.0 apare în 1999, TLS 1.1 în 2006, TLS 1.2 în 2008 și versiunea actuală, sigura și eficientă TLS 1.3 în 2018
- ▶ folosirea lui SSL 3.0, TLS 1.0 și TLS 1.1 trebuie evitată, toate trei prezintă probleme de securitate
- ▶ se recomandă folosirea lui TLS 1.3

TLS - Transport Layer Security

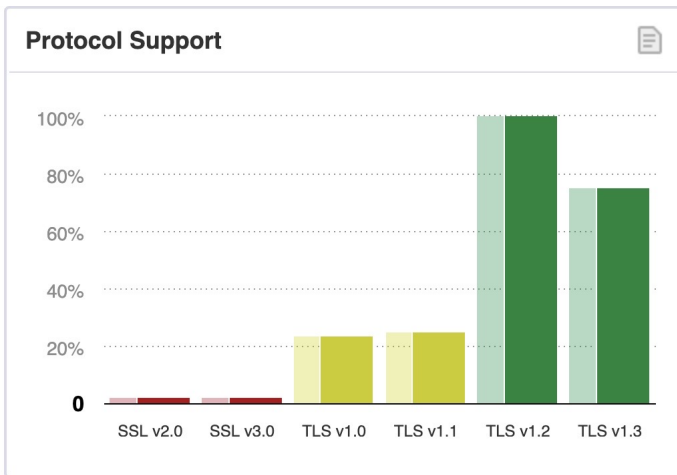


Figure: Acoperirea versiunilor de TLS conform SSL Pulse
(iunie 2025)

TLS - Transport Layer Security

- ▶ Protocolul TLS permite unui client (de ex. browser web) și unui server (de ex. website) să se pună de acord asupra unui set de chei pe care să le folosească ulterior pentru comunicare criptată și autentificare
- ▶ Protocolul TLS constă din 2 părți:
 1. *protocolul handshake* - realizează schimbul de chei care stabilește un set de chei comune
 2. *protocolul record-layer* - folosește cheile stabilite pentru criptare/autentificare ulterioară
- ▶ În continuare vom prezenta protocolul handshake din versiunea curentă TLS 1.3

Handshake protocol (TLS 1.3)...

Client



x secret

N_C nonce

$$K = g^{xy}$$

$$mk = KDF(K, N_C, N_B)$$

$$k_C, k'_C, k_S, k'_S = G(mk)$$

Banca



$pk, sk, cert_{CA \rightarrow banca}$

y secret N_B nonce

$$K = g^{xy}$$

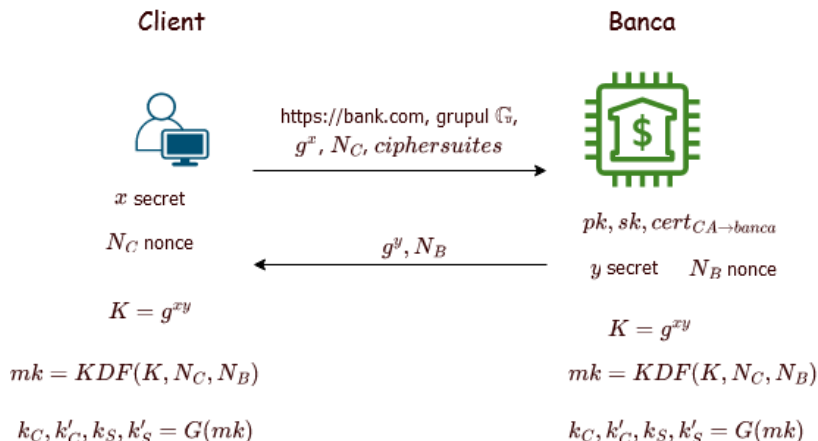
$$mk = KDF(K, N_C, N_B)$$

$$k_C, k'_C, k_S, k'_S = G(mk)$$

$\xrightarrow{\text{https://bank.com, grupul } \mathbb{G}, g^x, N_C, \text{ciphersuites}}$

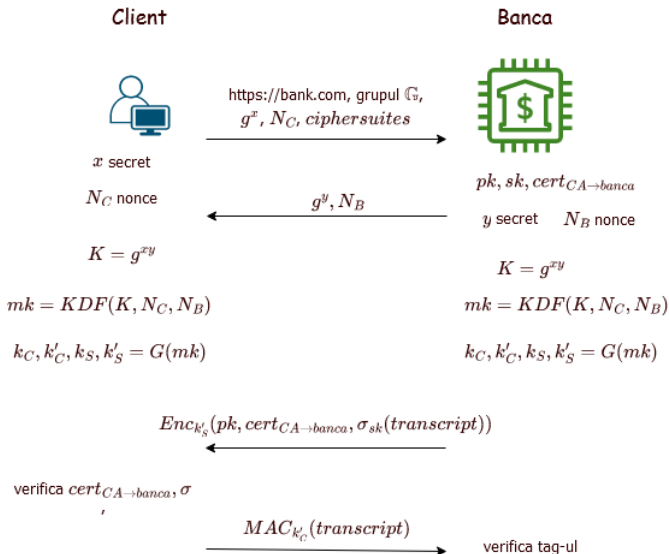
$\xleftarrow{g^y, N_B}$

Handshake protocol (TLS 1.3)...

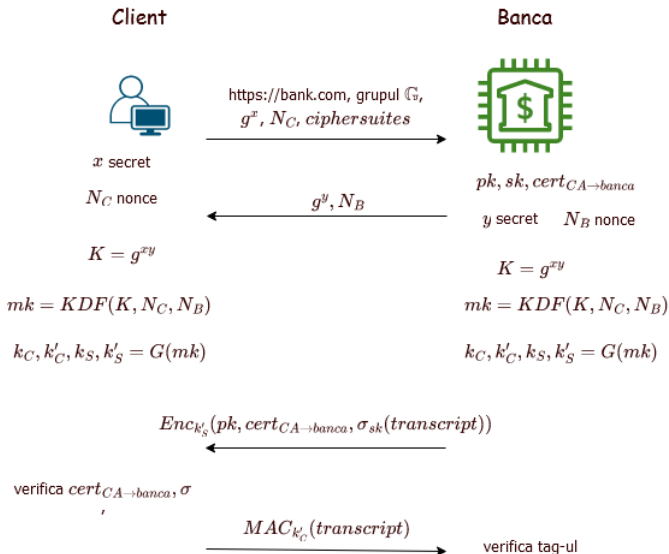


- ▶ KDF = key derivation function - algoritm criptografic pentru derivarea de chei, pe baza unui PRF
- ▶ G = generator de numere pseudo-aleatoare (PRG)

...Handshake protocol...



...Handshake protocol...



...Handshake protocol

- ▶ *ciphersuites* - colecție de algoritmi criptografici

...Handshake protocol

- ▶ *ciphersuites* - colecție de algoritmi criptografici
- ▶ TLS 1.3 suportă 5 ciphersuites:

...Handshake protocol

- ▶ *ciphersuites* - colecție de algoritmi criptografici
- ▶ TLS 1.3 suportă 5 ciphersuites:
 - ▶ TLS_AES_128_GCM_SHA256
 - ▶ TLS_AES_256_GCM_SHA384
 - ▶ TLS_CHACHA20_POLY1305_SHA256
 - ▶ TLS_AES_128_CCM_SHA256
 - ▶ TLS_AES_128_CCM_8_SHA256



...Handshake protocol

- ▶ *ciphersuites* - colecție de algoritmi criptografici
- ▶ TLS 1.3 suportă 5 ciphersuites:
 - ▶ TLS_AES_128_GCM_SHA256
 - ▶ TLS_AES_256_GCM_SHA384
 - ▶ TLS_CHACHA20_POLY1305_SHA256
 - ▶ TLS_AES_128_CCM_SHA256
 - ▶ TLS_AES_128_CCM_8_SHA256



- ▶ *transcript* - reprezintă mesajele trimise în cadrul protocolului până la momentul curent

...Handshake protocol

- ▶ *ciphersuites* - colecție de algoritmi criptografici
- ▶ TLS 1.3 suportă 5 ciphersuites:
 - ▶ TLS_AES_128_GCM_SHA256
 - ▶ TLS_AES_256_GCM_SHA384
 - ▶ TLS_CHACHA20_POLY1305_SHA256
 - ▶ TLS_AES_128_CCM_SHA256
 - ▶ TLS_AES_128_CCM_8_SHA256



- ▶ *transcript* - reprezintă mesajele trimise în cadrul protocolului până la momentul curent
- ▶ cheile k'_S și k'_C sunt folosite doar în handshake

TLS 1.3

- ▶ clientul și serverul, partajează, la finalul protocolului, cheile k_S și k_C , pe care le folosesc ulterior pentru criptare și autentificare

TLS 1.3

- ▶ clientul și serverul, partajează, la finalul protocolului, cheile k_S și k_C , pe care le folosesc ulterior pentru criptare și autentificare
- ▶ în plus, clientul are garanția că la final a partajat cheile cu server-ul legitim și că acestea nu au fost interceptate sau modificate de o terță parte

TLS 1.3

- ▶ clientul și serverul, partajează, la finalul protocolului, cheile k_S și k_C , pe care le folosesc ulterior pentru criptare și autentificare
- ▶ în plus, clientul are garanția că la final a partajat cheile cu server-ul legitim și că acestea nu au fost interceptate sau modificate de o terță parte **de ce?**

TLS 1.3

- ▶ clientul și serverul, partajează, la finalul protocolului, cheile k_S și k_C , pe care le folosesc ulterior pentru criptare și autentificare
- ▶ în plus, clientul are garanția că la final a partajat cheile cu server-ul legitim și că acestea nu au fost interceptate sau modificate de o terță parte de ce?
- ▶ în handshake-ul din TLS 1.2, clientul și server-ul foloseau o schemă de criptare cu cheie publică în locul schimbului de chei Diffie-Hellman

TLS 1.3

- ▶ clientul și serverul, partajează, la finalul protocolului, cheile k_S și k_C , pe care le folosesc ulterior pentru criptare și autentificare
- ▶ în plus, clientul are garanția că la final a partajat cheile cu server-ul legitim și că acestea nu au fost interceptate sau modificate de o terță parte **de ce?**
- ▶ în handshake-ul din TLS 1.2, clientul și server-ul foloseau o schemă de criptare cu cheie publică în locul schimbului de chei Diffie-Hellman
- ▶ clientul doar alegea o cheie K pe care o trimitea serverului criptată cu cheia lui publică

TLS 1.3

- ▶ aceasta a fost eliminată în TLS 1.3 pentru că nu asigură proprietatea de *forward secrecy* - care presupune că, în cazul compromiterii server-ului, cheile de sesiune anterioare nu sunt compromise

TLS 1.3

- ▶ aceasta a fost eliminată în TLS 1.3 pentru că nu asigură proprietatea de *forward secrecy* - care presupune că, în cazul compromiterii server-ului, cheile de sesiune anterioare nu sunt compromise
- ▶ în TLS 1.3, în ceea ce privește server-ul, cheia K este calculată pe baza secretului y în fiecare sesiune, secret care este unul nou de fiecare dată și care nu trebuie menținut între sesiuni; deci compromiterea server-ului (aflarea cheii lui secrete) nu duce la aflarea cheii K

TLS 1.3

- ▶ aceasta a fost eliminată în TLS 1.3 pentru că nu asigură proprietatea de *forward secrecy* - care presupune că, în cazul compromiterii server-ului, cheile de sesiune anterioare nu sunt compromise
- ▶ în TLS 1.3, în ceea ce privește server-ul, cheia K este calculată pe baza secretului y în fiecare sesiune, secret care este unul nou de fiecare dată și care nu trebuie menținut între sesiuni; deci compromiterea server-ului (aflarea cheii lui secrete) nu duce la aflarea cheii K
- ▶ în TLS 1.2, cheia secretă K ii este trimisă server-ului criptată cu cheia lui publică; compromiterea cheii secrete server-ului duce la compromiterea cheilor de sesiune din toate sesiunile

TLS 1.3

- ▶ aceasta a fost eliminată în TLS 1.3 pentru că nu asigură proprietatea de *forward secrecy* - care presupune că, în cazul compromiterii server-ului, cheile de sesiune anterioare nu sunt compromise
- ▶ în TLS 1.3, în ceea ce privește server-ul, cheia K este calculată pe baza secretului y în fiecare sesiune, secret care este unul nou de fiecare dată și care nu trebuie menținut între sesiuni; deci compromiterea server-ului (aflarea cheii lui secrete) nu duce la aflarea cheii K
- ▶ în TLS 1.2, cheia secretă K ii este trimisă server-ului criptată cu cheia lui publică; compromiterea cheii secrete server-ului duce la compromiterea cheilor de sesiune din toate sesiunile
- ▶ în protocolul *record*, clientul și server-ul comunică în mod confidențial și autentificat folosind o schemă de criptare
autentificată

TLS 1.3 vs. TLS 1.2

► număr redus de runde

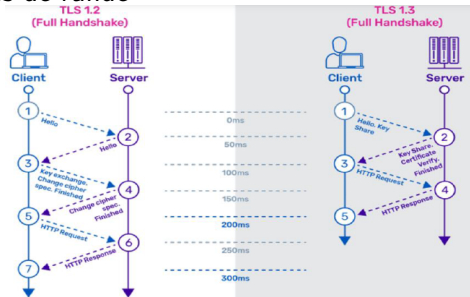


Figure: Sursa: www.a10networks.com

TLS 1.3 vs. TLS 1.2

- număr redus de runde

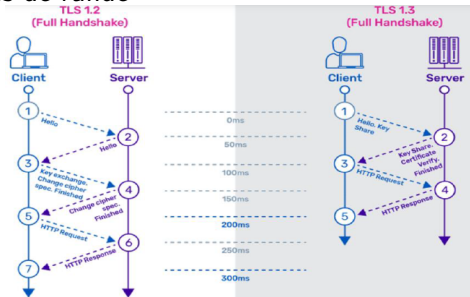


Figure: Sursa: www.a10networks.com

- eliminarea algoritmilor criptografici vulnerabili precum SHA-1, RC4, DES, 3DES, AES-CBC, MD5

TLS 1.3 vs. TLS 1.2

- număr redus de runde

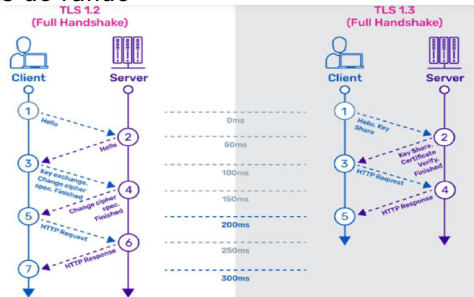


Figure: Sursa: www.a10networks.com

- eliminarea algoritmilor criptografici vulnerabili precum SHA-1, RC4, DES, 3DES, AES-CBC, MD5
- 0-RTT (zero round-trip) – reluarea unei sesiuni mai vechi cu un website e mult mai rapida